removed. But to our surprise, we're then presented with the login screen instead. We just deleted the only administrative user from the system. When the next request came in, the authentication failed, so the application refused to let us in. We have to log in again before using any administrative functions.

But now we have an embarrassing problem: there are no administrative users in the database, so we can't log in.

Fortunately, we can quickly add a user to the database from the command line. If you invoke the `rails console` command, Rails invokes Ruby's `irb` utility, but it does so in the context of your Rails application. That means you can interact with your application's code by typing Ruby statements and looking at the values they return.

We can use this to invoke our user model directly, having it add a user into the database for us:

```
depot> bin/rails console
Loading development environment.
>> User.create(name: 'dave', password: 'secret', password_confirmation: 'secret')
=> #<User:0x2933060 @attributes={...} ... >
>> User.count
=> 1
```

The `>>` sequences are prompts. After the first, we call the `User` class to create a new user, and after the second, we call it again to show that we do indeed have a single user in our database. After each command we enter, `rails console` displays the value returned by the code (in the first case, it's the model object, and in the second case, it's the count).

Panic over. We can now log back in to the application. But how can we stop this from happening again? We have several ways. For example, we could write code that prevents you from deleting your own user. That doesn't quite work: in theory, A could delete B at just the same time that B deletes A. Let's try a different approach. We'll delete the user inside a database transaction. Transactions provide an all-or-nothing proposition, stating that each work unit performed in a database must either complete in its entirety or none of them will